



SNA-GraphSAGE

Social Network Analysis with Graph Neural Networks

Part 1 of 2 — **Research & Results Walkthrough**

The approach, the experiments, and the measured numbers.

Python

PyTorch Geometric

GraphSAGE

scikit-learn

NetworkX

Prepared 14 July 2026 · Author: Bhanu Prakash

o • What this project is

A research framework for **Social Network Analysis (SNA) using Graph Neural Networks**, built around **GraphSAGE**. It reproduces and extends the paper "*Social Network Analysis Based on GraphSAGE*" (Xiao, Wu, Wang — ISCID 2019, shipped as [MPResearch.pdf](#)) with a modern multi-architecture GNN zoo, classic SNA centrality features, a degree-adaptive neighbour sampler, and two extra graph tasks. It is a CPU-runnable, multi-seed, methodologically rigorous experiment harness — not a deployed app.

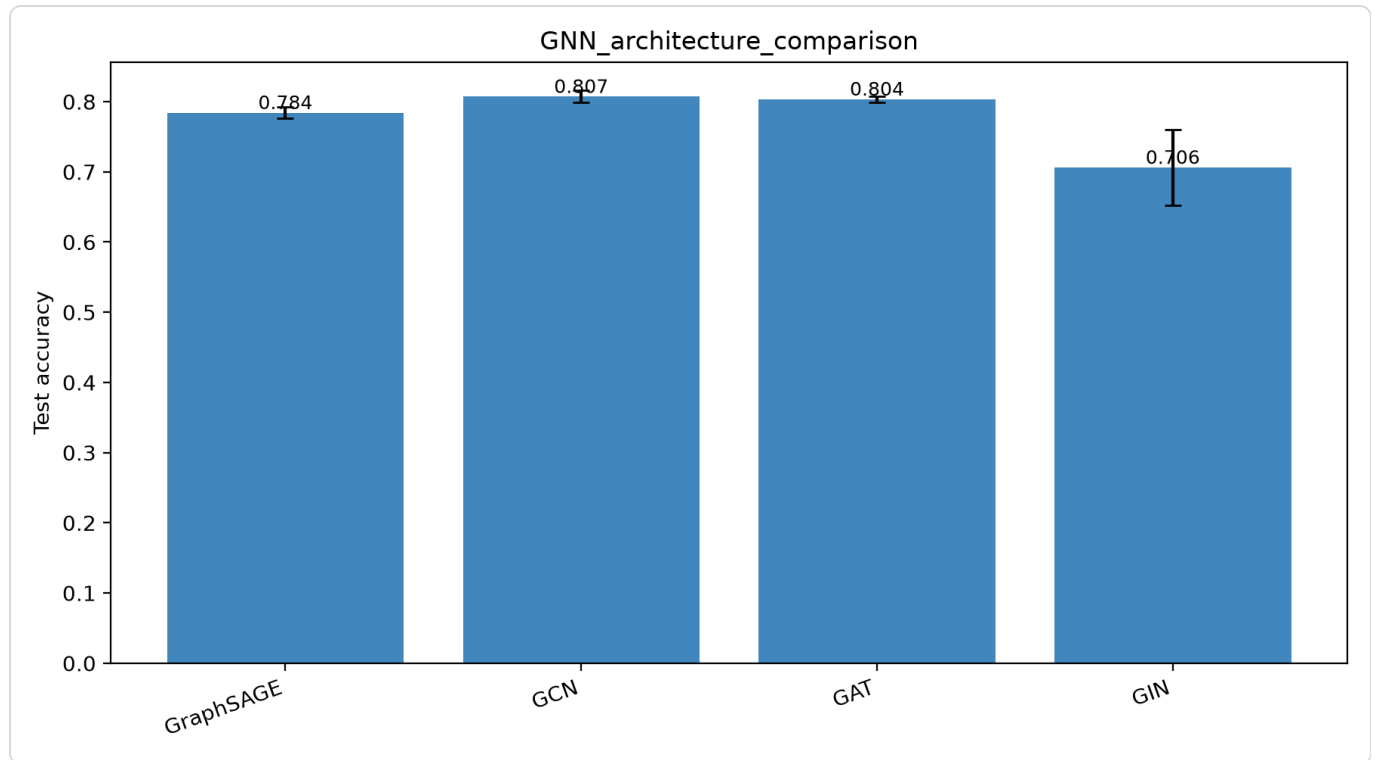
The interviewer one-liner: "It studies how an inductive GNN (GraphSAGE) fuses node content and network structure on citation graphs — with a 4-architecture zoo (SAGE/GCN/GAT/GIN), an ablation of eight injected SNA centrality features, a depth/oversmoothing study, a degree-adaptive neighbour sampler, and added link-prediction and community-detection tasks — all evaluated with validation-based selection and mean±std over three seeds."

Stack & models

COMPONENT	CHOICE
GNN framework	PyTorch Geometric (SAGEConv, GCNConv, GATConv, GINConv, Planetoid, RandomLinkSplit)
Models	Unified GNN class (sage/gcn/gat/gin); GraphSAGE encoder + dot-product link decoder; DeepWalk (Word2Vec) & Logistic-Regression baselines
Classic SNA / analysis	NetworkX (centralities, Louvain), scikit-learn (LR, KMeans, PCA, t-SNE, metrics), gensim (DeepWalk)
Datasets	Planetoid citation graphs — Cora (default), CiteSeer, PubMed
Compute	CPU-friendly (auto-CUDA if present); a full Cora run is ~3–4 min

1 • Experiments & results (Cora, 3 seeds)

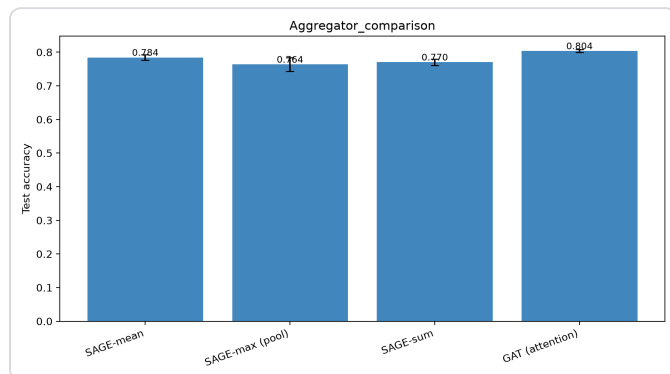
Architecture comparison — SAGE vs GCN vs GAT vs GIN



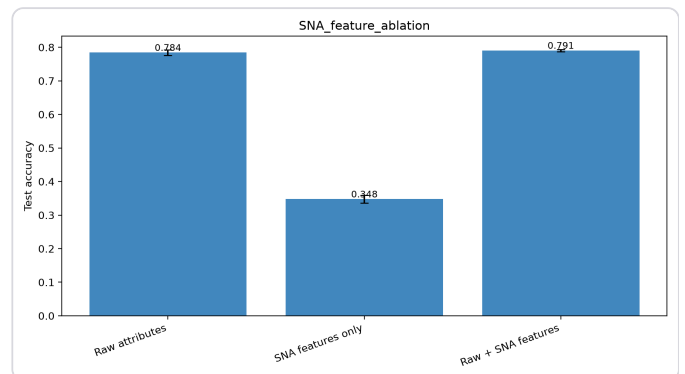
Node-classification test accuracy across the four GNN architectures (mean $\pm$ std over seeds 42/43/44).

All four run under one unified model class with validation-based checkpoint selection and early stopping. Results: LR baseline 0.576 · DeepWalk+LR 0.696 · **GraphSAGE 0.784** · GIN 0.706 · GAT 0.804 · **GCN 0.807 (best on Cora)**. GraphSAGE is the framework's inductive centrepiece; GCN happens to edge it out on this transductive benchmark.

Aggregator study & SNA feature ablation



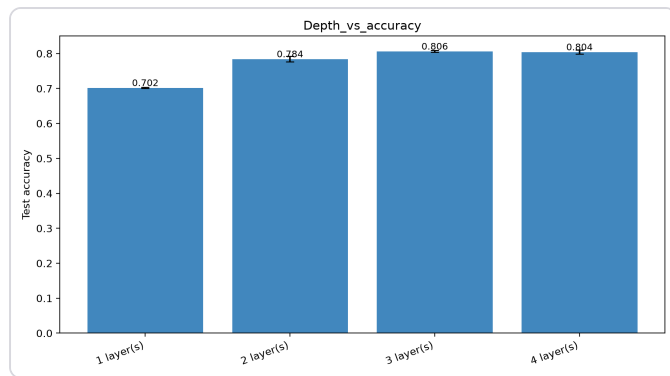
SAGE mean / max / sum vs GAT attention.



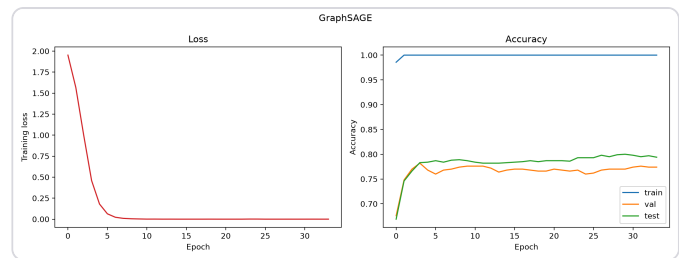
Raw vs SNA-only vs raw+SNA features.

Aggregators: SAGE-mean 0.784 · SAGE-max 0.764 · SAGE-sum 0.770 · GAT-attention 0.804 — the modern analogue of the paper's mean/pool/LSTM comparison. Feature ablation injects **eight z-scored SNA centralities** (degree, log-degree, clustering coefficient, PageRank, approximate betweenness, eigenvector centrality, k-core, avg-neighbour-degree): SNA-only 0.348 · raw 0.784 · **raw+SNA 0.791** — the structural features add +0.7 pts and lower variance.

Depth (oversmoothing) & degree-adaptive sampling



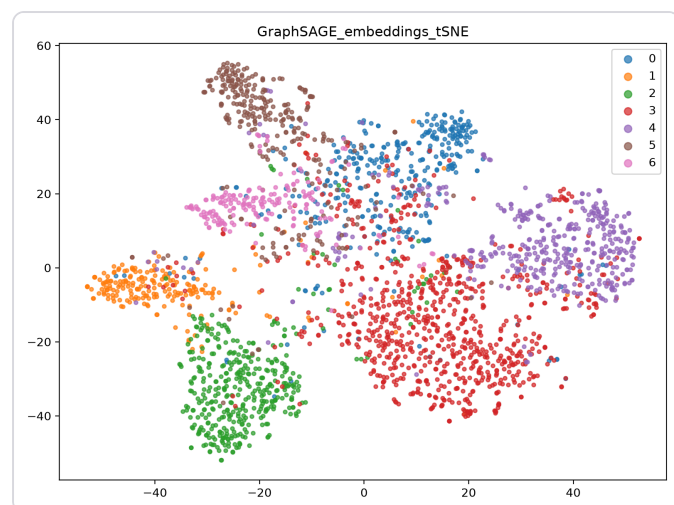
1–4 message-passing layers.



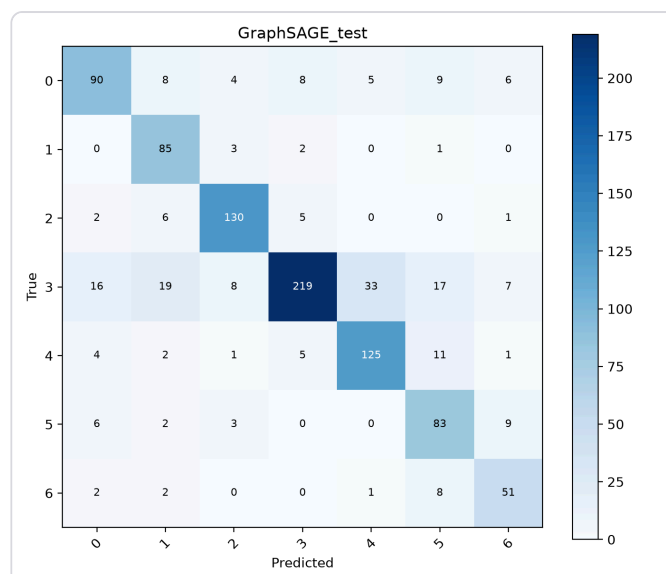
GraphSAGE training curves (loss + train/val/test acc).

Depth: 1L 0.702 · 2L 0.784 · **3L 0.806** · 4L 0.804 (accuracy plateaus/degrades — oversmoothing). The hand-rolled **degree-adaptive neighbour sampler** ($S_i = \text{clip}(c + \lfloor \text{deg} \cdot w \rfloor, 1, \text{cap})$, the paper's rule re-scaled for low-degree citation graphs) beats fixed fan-out: fixed 0.799 vs **adaptive 0.819** (+2 pts).

Learned embeddings & confusion



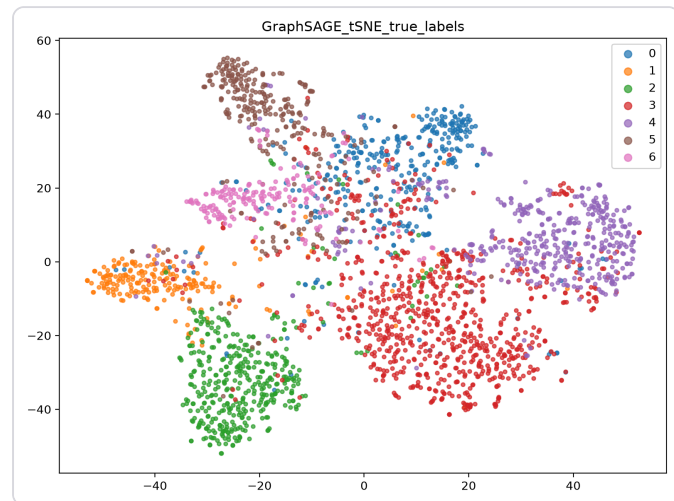
t-SNE of the learned GraphSAGE embeddings.



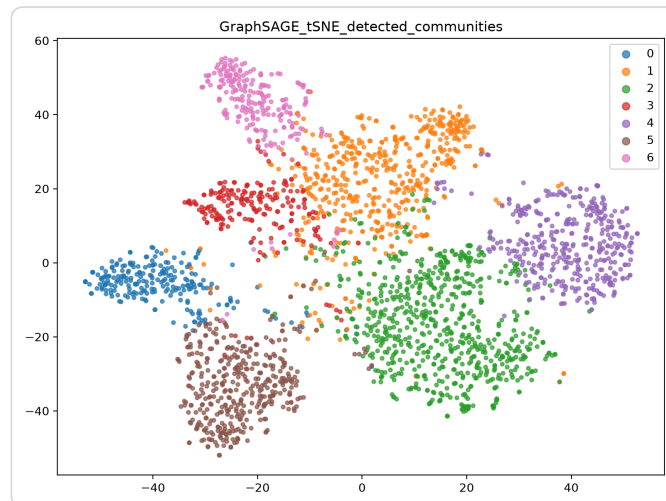
Test-set confusion matrix.

The t-SNE projection shows the seven Cora classes separating cleanly in the learned space — visual evidence that the embeddings capture class structure. Baseline embeddings (DeepWalk, raw features) are also plotted for comparison.

Link prediction & community detection



t-SNE coloured by true class.



t-SNE coloured by detected community.

- **Link prediction** — GraphSAGE encoder + dot-product decoder with fresh per-epoch negative sampling: test **AUC 0.831, AP 0.835** (fixing negatives instead drops AUC to ~ 0.73 — a documented pitfall).
- **Community detection** — Louvain (structure) NMI 0.448 · KMeans on raw features NMI 0.147 · **KMeans on learned embeddings NMI 0.574 / ARI 0.559** — learned embeddings beat both pure-structure and pure-content clustering.

Contributions over the reference paper: (1) a 4-architecture zoo; (2) injection + ablation of 8 classic SNA centralities; (3) a re-scaled degree-adaptive sampler for low-degree graphs; (4) two added tasks

(link prediction, community detection); (5) a rigorous protocol — validation-based selection, early stopping, 3-seed mean $\pm$ std, fresh per-epoch negatives. Written up in [Enhanced_SNA_GNN_Paper.pdf](#) .

2 · Likely interview questions

Q: What makes GraphSAGE "inductive"?

It learns aggregator functions over a node's sampled neighbourhood rather than a fixed embedding per node, so it can generate embeddings for nodes (or graphs) unseen at training time — unlike transductive methods such as vanilla DeepWalk.

Q: Why inject classic SNA centralities if the GNN already sees structure?

The GNN sees local neighbourhood structure via message passing, but global positional signals (PageRank, betweenness, k-core) aren't directly available in a few hops. Concatenating them as node features gives a small, reliable accuracy bump and lower variance (raw+SNA 0.791 vs raw 0.784).

Q: How did you avoid over-optimistic results?

Checkpoint on validation accuracy (never test), early stopping with patience, report mean±std over 3 seeds, and for link prediction resample negatives every epoch — fixing negatives inflates AUC by ~0.10.

Q: What is oversmoothing and did you see it?

Stacking too many message-passing layers makes node representations converge to indistinguishable values. The depth study shows accuracy peaking at 3 layers (0.806) then dipping at 4 — the classic oversmoothing signature.

Q: Why does the degree-adaptive sampler help?

High-degree hubs need more neighbour samples to summarise; low-degree nodes need fewer. Scaling the sample size with node degree (clipped) captures more signal where it exists and saves compute elsewhere — +2 pts over fixed fan-out on Cora.